

Ex8 Report | Building a Web API

Nguyen Tieu Phuong 20210692

Source code

See the attached source code with this submission.

This web API is built using NodeJS and MongoDB.

The project is structured as follows:

/project-folder

```
|-- /models
|   |-- sensorData.js
|   |-- user.js
|-- /routes
|   |-- sensorRoutes.js
|   |-- userRoutes.js
|-- app.js
|-- swagger.js (optional)
```

Models

I have 2 models for the sensor data and for user. We specify them as how they are described in the problem statement.

sensorData.js

```
const mongoose = require('mongoose');

const sensorDataSchema = new mongoose.Schema({
  Id: { type: Number, required: true },
  SensorName: { type: String, required: true },
  SensorValue: { type: Number, required: true },
})

module.exports = mongoose.model('SensorData', sensorDataSchema);
```

user.js

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  Id: { type: Number, required: true }, // Custom Id field
  UserName: { type: String, required: true },
  Password: { type: String, required: true },
  Email: { type: String, required: true },
});

module.exports = mongoose.model('User', userSchema);
```

Routes

We need to define the route (aka. Endpoints) for interacting with this API. Here I also include the Swagger documentation, but that's optional.

sensorRoutes.js

```
const express = require('express');
const router = express.Router();
const SensorData = require('../models/sensorData');

/**
 * @swagger
 * tags:
 *   name: SensorDatas
 *   description: API for managing SensorData
 */

/**
 * @swagger
 * /api/SensorDatas:
 *   get:
 *     summary: Get all SensorDatas
 *     tags: [SensorDatas]
 *     responses:
 *       200:
 *         description: A list of SensorDatas
 *         content:
 *           application/json:
 *             schema:
 *               type: array
 *               items:
 *                 $ref: '#/components/schemas/SensorData'
 */
```

```

router.get('/', async (req, res) => {
  try {
    const sensors = await SensorData.find();
    res.json(sensors);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

/**
 * @swagger
 * /api/SensorDatas:
 *   post:
 *     summary: Create a new SensorData
 *     tags: [SensorDatas]
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             $ref: '#/components/schemas/SensorData'
 *     responses:
 *       201:
 *         description: SensorData created successfully
 *         content:
 *           application/json:
 *             schema:
 *               $ref: '#/components/schemas/SensorData'
 *       400:
 *         description: Bad Request
 */

router.post('/', async (req, res) => {
  const sensor = new SensorData({
    Id: req.body.Id, // Ensure unique ID
    SensorName: req.body.SensorName,
    SensorValue: req.body.SensorValue,
  });

  try {
    const newSensor = await sensor.save();
    res.status(201).json(newSensor);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

```

```

/**
 * @swagger
 * /api/SensorDatas/{id}:
 *   get:
 *     summary: Get a SensorData by ID
 *     tags: [SensorDatas]
 *     parameters:
 *       - in: path
 *         name: id
 *         required: true
 *         schema:
 *           type: integer
 *         description: The SensorData ID
 *     responses:
 *       200:
 *         description: SensorData retrieved successfully
 *         content:
 *           application/json:
 *             schema:
 *               $ref: '#/components/schemas/SensorData'
 *       404:
 *         description: SensorData not found
 */
router.get('/:id', async (req, res) => {
  try {
    const sensor = await SensorData.findOne({ Id: req.params.id });
    if (!sensor) return res.status(404).json({ message: 'Sensor not
found' });
    res.json(sensor);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

/**
 * @swagger
 * /api/SensorDatas/{id}:
 *   put:
 *     summary: Update a SensorData by ID
 *     tags: [SensorDatas]
 *     parameters:
 *       - in: path
 *         name: id
 *         required: true
 *         schema:
 *           type: integer
 *         description: The SensorData ID
 *     requestBody:

```

```

*     required: true
*     content:
*       application/json:
*         schema:
*           $ref: '#/components/schemas/SensorData'
*   responses:
*     200:
*       description: SensorData updated successfully
*       content:
*         application/json:
*           schema:
*             $ref: '#/components/schemas/SensorData'
*     404:
*       description: SensorData not found
*     400:
*       description: Bad Request
*/
router.put('/:id', async (req, res) => {
  try {
    const updatedSensor = await SensorData.findOneAndUpdate(
      { Id: req.params.id },
      req.body,
      { new: true }
    );
    if (!updatedSensor) return res.status(404).json({ message: 'Sensor
not found' });
    res.json(updatedSensor);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

/**
* @swagger
* /api/SensorDatas/{id}:
*   delete:
*     summary: Delete a SensorData by ID
*     tags: [SensorDatas]
*     parameters:
*       - in: path
*         name: id
*         required: true
*         schema:
*           type: integer
*         description: The SensorData ID
*     responses:
*       200:
*         description: SensorData deleted successfully

```

```

*      404:
*      description: SensorData not found
*/
router.delete('/:id', async (req, res) => {
  try {
    const sensor = await SensorData.findOneAndDelete({ Id:
req.params.id });
    if (!sensor) return res.status(404).json({ message: 'Sensor not
found' });
    res.json({ message: 'Sensor deleted' });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

module.exports = router;

```

userRoutes.js

```

const express = require('express');
const router = express.Router();
const User = require('../models/user');

/**
 * @swagger
 * tags:
 *   name: Users
 *   description: API for managing Users
 */

/**
 * @swagger
 * /api/Users:
 *   get:
 *     summary: Get all Users
 *     tags: [Users]
 *     responses:
 *       200:
 *         description: A list of Users
 *         content:
 *           application/json:
 *             schema:
 *               type: array
 *               items:
 *                 $ref: '#/components/schemas/User'
 */

```

```
router.get('/', async (req, res) => {
  try {
    const users = await User.find();
    res.json(users);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
```

```
/**
 * @swagger
 * /api/Users:
 *   post:
 *     summary: Create a new User
 *     tags: [Users]
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             $ref: '#/components/schemas/User'
 *     responses:
 *       201:
 *         description: User created successfully
 *         content:
 *           application/json:
 *             schema:
 *               $ref: '#/components/schemas/User'
 *       400:
 *         description: Bad Request
 */
```

```
router.post('/', async (req, res) => {
  const user = new User({
    Id: req.body.Id, // Ensure unique ID
    UserName: req.body.UserName,
    Password: req.body.Password,
    Email: req.body.Email,
  });

  try {
    const newUser = await user.save();
    res.status(201).json(newUser);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});
```

```
/**
```

```

* @swagger
* /api/Users/{id}:
*   get:
*     summary: Get a User by ID
*     tags: [Users]
*     parameters:
*       - in: path
*         name: id
*         required: true
*         schema:
*           type: integer
*         description: The User ID
*     responses:
*       200:
*         description: User retrieved successfully
*         content:
*           application/json:
*             schema:
*               $ref: '#/components/schemas/User'
*       404:
*         description: User not found
*/
router.get('/:id', async (req, res) => {
  try {
    const user = await User.findOne({ Id: req.params.id });
    if (!user) return res.status(404).json({ message: 'User not
found' });
    res.json(user);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
/**
* @swagger
* /api/Users/{id}:
*   put:
*     summary: Update a User by ID
*     tags: [Users]
*     parameters:
*       - in: path
*         name: id
*         required: true
*         schema:
*           type: integer
*         description: The User ID
*     requestBody:
*       required: true

```



```

*       content:
*       application/json:
*       schema:
*       $ref: '#/components/schemas/User'
*   responses:
*       200:
*       description: User updated successfully
*       content:
*       application/json:
*       schema:
*       $ref: '#/components/schemas/User'
*       404:
*       description: User not found
*       400:
*       description: Bad Request
*/
router.put('/:id', async (req, res) => {
  try {
    const updatedUser = await User.findOneAndUpdate(
      { Id: req.params.id },
      req.body,
      { new: true }
    );
    if (!updatedUser) return res.status(404).json({ message: 'User not
found' });
    res.json(updatedUser);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

/**
 * @swagger
 * /api/Users/{id}:
 *   delete:
 *     summary: Delete a User by ID
 *     tags: [Users]
 *     parameters:
 *       - in: path
 *         name: id
 *         required: true
 *         schema:
 *           type: integer
 *         description: The User ID
 *     responses:
 *       200:
 *       description: User deleted successfully
 *       404:

```

```

*           description: User not found
*/
router.delete('/:id', async (req, res) => {
  try {
    const user = await User.findOneAndDelete({ Id: req.params.id });
    if (!user) return res.status(404).json({ message: 'User not
found' });
    res.json({ message: 'User deleted' });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

module.exports = router;

```

Main Application

App.js

```

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const { swaggerUi, specs } = require('./swagger');
const sensorRoutes = require('./routes/sensorRoutes');
const userRoutes = require('./routes/userRoutes');

const app = express();

// Middleware
app.use(cors());
app.use(express.json());

// MongoDB Connection
const mongoURI = 'mongodb+srv://tieuphuong:jIA8tkwj4AjIPois@ex8-
tieuphuong.jwerc.mongodb.net/?retryWrites=true&w=majority&appName=Ex8-
TieuPhuong';
mongoose
  .connect(mongoURI)
  .then(() => console.log('Connected to MongoDB'))
  .catch(err => console.error('Error connecting to MongoDB', err));

// Swagger UI
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(specs));

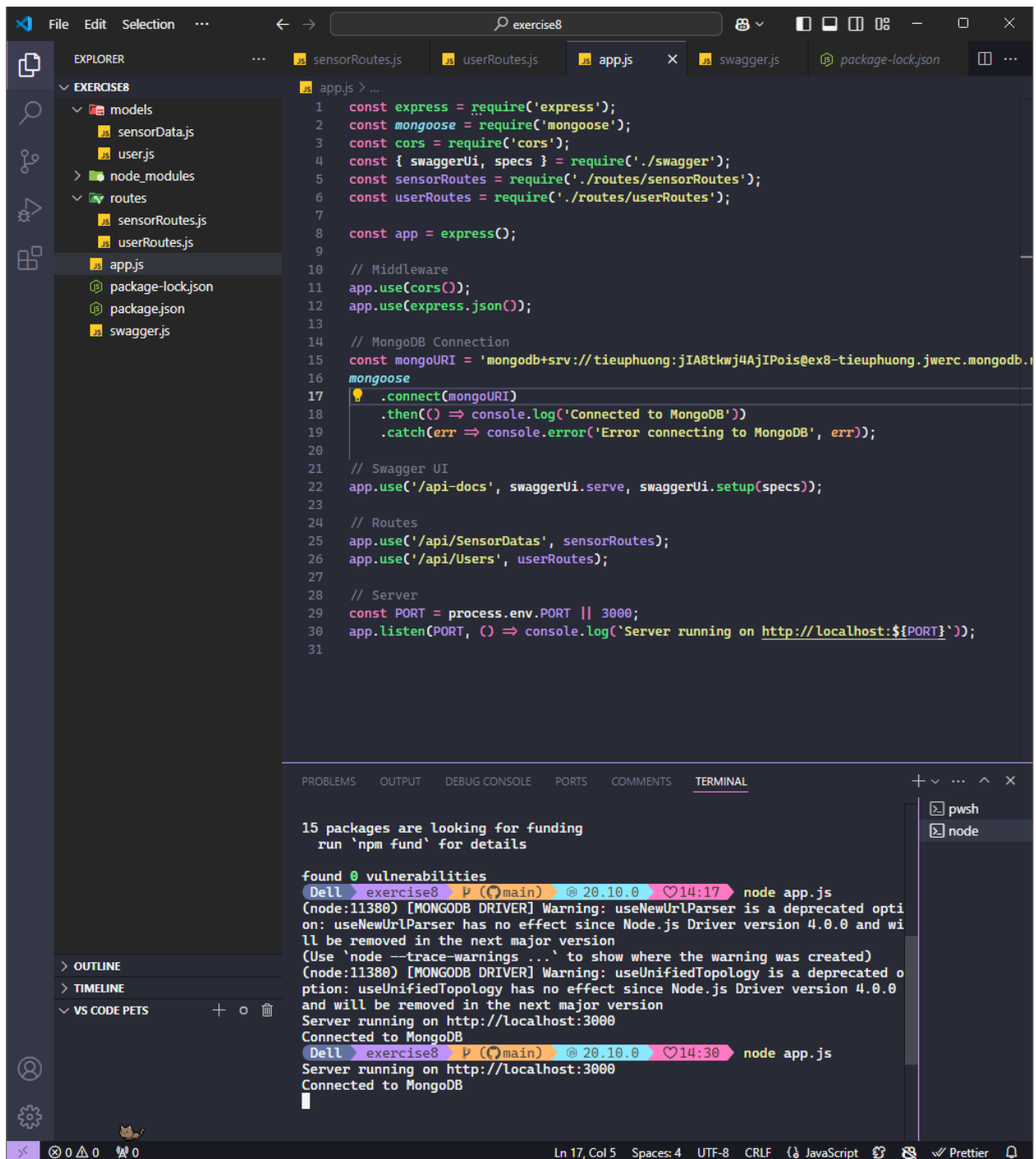
// Routes
app.use('/api/SensorDatas', sensorRoutes);

```

```
app.use('/api/Users', userRoutes);

// Server
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Server running on
http://localhost:${PORT}`));
```

Running the Web API



The screenshot shows a Visual Studio Code editor with a project named 'EXERCISE8'. The Explorer sidebar on the left shows the file structure, including 'models', 'node_modules', 'routes', and 'app.js'. The main editor displays the 'app.js' file, which is a Node.js application. The code includes imports for 'express', 'mongoose', 'cors', and 'swagger', and defines routes for 'sensorRoutes' and 'userRoutes'. The application is configured to run on port 3000 and connect to a MongoDB database. The terminal at the bottom shows the output of running 'node app.js', which includes warnings about deprecated options and confirms that the server is running on http://localhost:3000 and connected to MongoDB.

```
1 const express = require('express');
2 const mongoose = require('mongoose');
3 const cors = require('cors');
4 const { swaggerUi, specs } = require('./swagger');
5 const sensorRoutes = require('./routes/sensorRoutes');
6 const userRoutes = require('./routes/userRoutes');
7
8 const app = express();
9
10 // Middleware
11 app.use(cors());
12 app.use(express.json());
13
14 // MongoDB Connection
15 const mongoURI = 'mongodb+srv://tieuphuong:jIA8tkwj4AjIPois@ex8-tieuphuong.jwerc.mongodb.net';
16 mongoose
17   .connect(mongoURI)
18   .then(() => console.log('Connected to MongoDB'))
19   .catch(err => console.error('Error connecting to MongoDB', err));
20
21 // Swagger UI
22 app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(specs));
23
24 // Routes
25 app.use('/api/SensorDatas', sensorRoutes);
26 app.use('/api/Users', userRoutes);
27
28 // Server
29 const PORT = process.env.PORT || 3000;
30 app.listen(PORT, () => console.log('Server running on http://localhost:${PORT}'));
31
```

15 packages are looking for funding
run 'npm fund' for details

found 0 vulnerabilities

Dell exercise8 (main) @ 20.10.0 14:17 node app.js
(node:11380) [MONGODB DRIVER] Warning: useUrlParser is a deprecated option: useUrlParser has no effect since Node.js Driver version 4.0.0 and will be removed in the next major version
(Use 'node --trace-warnings ...' to show where the warning was created)
(node:11380) [MONGODB DRIVER] Warning: useUnifiedTopology is a deprecated option: useUnifiedTopology has no effect since Node.js Driver version 4.0.0 and will be removed in the next major version
Server running on http://localhost:3000
Connected to MongoDB

Dell exercise8 (main) @ 20.10.0 14:30 node app.js
Server running on http://localhost:3000
Connected to MongoDB

Server is running on port 3000.

Testing the API

Note: MongoDB comes with a default `_id` field, auto generated string (not incremental). For this exercise, I create another, my own `Id` field that serves as the id for the users and sensor data.

SensorDatas

The screenshot displays the MongoDB Atlas web interface. The top navigation bar includes the 'Ex8-TieuPhuong' logo, version '8.0.3', and region 'AWS Hong Kong (ap-east-1)'. The 'Collections' tab is selected, showing a list of databases and collections. The 'test' database is expanded, showing the 'sensordatas' collection. The 'test.sensordatas' collection page shows storage and document statistics. A filter bar is present with a query input field. Below the filter, the query results are displayed, showing three documents with fields: `_id`, `Id`, `SensorName`, `SensorValue`, and `__v`.

Document	<code>_id</code>	<code>Id</code>	<code>SensorName</code>	<code>SensorValue</code>	<code>__v</code>
1	<code>ObjectId('674d62bf9af6fa1fbb80a29c')</code>	<code>1</code>	<code>"Temperature"</code>	<code>-7</code>	<code>0</code>
2	<code>ObjectId('674d634c9af6fa1fbb80a2a0')</code>	<code>3</code>	<code>"Moisture"</code>	<code>85</code>	<code>0</code>
3	<code>ObjectId('674d63559af6fa1fbb80a2a2')</code>	<code>4</code>	<code>"Wind Speed"</code>	<code>12</code>	<code>0</code>

SensorDatas, as shown in MongoDB Atlas.

POST /api/SensorDatas

The screenshot displays the Postman API client interface. The top navigation bar includes 'Home', 'Workspaces', and 'API Network'. The left sidebar shows 'Collections', 'Environments', 'History', and a 'Collections' icon. The main area shows a POST request to 'http://localhost:3000/api/SensorDatas'. The request body is a JSON object with the following structure:

```
1 {
2   "Id": 1,
3   "SensorName": "Temperature",
4   "SensorValue": 20
5 }
```

The response is a 201 Created status with a JSON object containing the following structure:

```
1 {
2   "Id": 1,
3   "SensorName": "Temperature",
4   "SensorValue": 20,
5   "_id": "674d62bf9af6fa1fbb80a29c",
6   "__v": 0
7 }
```

The bottom status bar shows 'Online', 'Find and replace', 'Console', 'Postbot', 'Runner', 'Start Proxy', 'Cookies', 'Vault', 'Trash', and a help icon.

GET /api/SensorDatas

The screenshot shows the Postman API client interface. The top bar includes navigation links (Home, Workspaces, API Network), a search bar, and utility buttons (Invite, Upgrade). The left sidebar contains icons for Collections, Environments, History, and a workspace icon.

The main workspace displays a GET request to `http://localhost:3000/api/SensorDatas/`. The request is configured with the following settings:

- Method: GET
- URL: `http://localhost:3000/api/SensorDatas/`
- Params: none (selected)
- Authorization: none
- Headers: 6
- Body: none (selected)
- Scripts: none
- Settings: none

The response is displayed in the bottom panel, showing a status of **200 OK** with a response time of 96 ms and a body size of 548 B. The response body is formatted as JSON:

```
[{"_id": "674d62bf9af6fa1fbb80a29c", "Id": 1, "SensorName": "Temperature", "SensorValue": -7, "__v": 0}, {"_id": "674d634c9af6fa1fbb80a2a0", "Id": 3, "SensorName": "Moisture", "SensorValue": 85, "__v": 0}, {"_id": "674d63659af6fa1fbb80a2a2", "Id": 4, "SensorName": "Wind Speed", "SensorValue": 12, "__v": 0}]
```

The bottom status bar includes links for Online, Find and replace, Console, Postbot, Runner, Start Proxy, Cookies, Vault, Trash, and a help icon.

GET /api/SensorDatas/{id}

The screenshot displays the Postman API client interface. The top bar shows the 'API Network' tab with a search bar and an 'Upgrade' button. The left sidebar contains navigation icons for Overview, Collections, Environments, History, and a 'Left Tab' label. The main workspace shows a GET request to `http://localhost:3000/api/SensorDatas/1`. The 'Send' button is visible. Below the request bar, the 'Body' tab is selected, showing a '200 OK' status, a response time of 97 ms, and a body size of 360 B. The response body is displayed in JSON format, showing a single sensor data entry.

Request:

```
GET http://localhost:3000/api/SensorDatas/1
```

Response:

```
{
  "_id": "674d62bf9af6fa1fbb80a29c",
  "Id": 1,
  "SensorName": "Temperature",
  "SensorValue": 20,
  "__v": 0
}
```


PUT /api/SensorDatas/{id}

The screenshot shows the Postman API client interface. The top bar includes navigation links (Home, Workspaces, API Network), a search bar, and an 'Upgrade' button. The left sidebar contains icons for Collections, Environments, History, and a workspace icon. The main area displays a PUT request to `http://localhost:3000/api/SensorDatas/1`. The request body is a JSON object:

```
{  "SensorValue": -7}
```

. The response is a 200 OK status with a JSON object:

```
{  "_id": "674d62bf9af6fa1fbb80a29c",  "Id": 1,  "SensorName": "Temperature",  "SensorValue": -7,  "__v": 0}
```

. The bottom status bar shows 'Online', 'Find and replace', 'Console', 'Postbot', 'Runner', 'Start Proxy', 'Cookies', 'Vault', 'Trash', and a help icon.

Change temperature value, for example.

DELETE /api/SensorDatas/{id}

The screenshot displays the Postman API client interface. The top bar shows the 'API Network' tab with a search bar and an 'Upgrade' button. The left sidebar contains navigation icons for Overview, Collections, Environments, History, and a workspace icon. The main area shows a DELETE request to `http://localhost:3000/api/SensorDatas/2`. The request body is empty, and the 'raw' tab is selected. The response is a 200 OK status with a response time of 99 ms and a body size of 295 B. The response body is displayed in the 'Pretty' tab, showing a JSON object: `{\"message\": \"Sensor deleted\"}`. The bottom status bar includes links to Online, Find and replace, Console, Postbot, Runner, Start Proxy, Cookies, Vault, Trash, and a help icon.

Overview GET Get authentication GET https://api.thir GET https://api.thir GET https://api.thir exercise1 DEL http://localhost No environment

http://localhost:3000/api/SensorDatas/2 Save Share

DELETE http://localhost:3000/api/SensorDatas/2 Send

Params Authorization Headers (6) Body Scripts Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary GraphQL JSON

1

Body Cookies Headers (8) Test Results

200 OK 99 ms 295 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "Sensor deleted"
3 }
```

Online Find and replace Console Postbot Runner Start Proxy Cookies Vault Trash

Users

DATABASES: 1 COLLECTIONS: 2

VISUALIZE YOUR DATAREFRESH

+ Create Database

Search Namespaces

test

sensordatas

users

test.users

STORAGE SIZE: 36KB LOGICAL DATA SIZE: 237B TOTAL DOCUMENTS: 2 INDEXES TOTAL SIZE: 36KB

FindIndexesSchema Anti-PatternsAggregationSearch Indexes

Generate queries from natural language in Compass

INSERT DOCUMENT

FilterType a query: { field: 'value' }

Reset

Apply

Options

QUERY RESULTS: 1-2 OF 2

```
_id: ObjectId('674d661d9af6fa1fbb80a2ae')
Id: 1
UserName: "tieu"
Password: "qwerty"
Email: "tieu@aol.com"
__v: 0
```

```
_id: ObjectId('674d66739af6fa1fbb80a2b2')
Id: 2
UserName: "nguyen"
Password: "IknowThisIsNotAStrongPassword"
Email: "nguyen@outlook.com"
__v: 0
```

GET /api/Users

The screenshot displays the Postman interface for a GET request to `http://localhost:3000/api/Users/`. The request is configured with the method `GET` and the URL `http://localhost:3000/api/Users/`. The response is a `200 OK` status with a response time of `93 ms` and a size of `645 B`. The response body is a JSON array of three user objects, displayed in the `Body` tab with the `JSON` format selected.

Request Details:

- Method: `GET`
- URL: `http://localhost:3000/api/Users/`
- Params: none
- Authorization: none
- Headers: 6
- Body: This request does not have a body

Response Details:

- Status: `200 OK`
- Time: `93 ms`
- Size: `645 B`
- Save Response: [icon]

Response Body (JSON):

```
[
  {
    "_id": "674d661d9af6fa1fbb80a2ae",
    "Id": 1,
    "UserName": "tieu",
    "Password": "qwerty",
    "Email": "tieu@aol.com",
    "__v": 0
  },
  {
    "_id": "674d664a9af6fa1fbb80a2b0",
    "Id": 2,
    "UserName": "phuong",
    "Password": "iloveyou123",
    "Email": "phuong@gmail.com",
    "__v": 0
  },
  {
    "_id": "674d66739af6fa1fbb80a2b2",
    "Id": 3,
    "UserName": "nguyen",
    "Password": "IKnowThisIsNotAStrongPassword",
    "Email": "nguyen@outlook.com",
    "__v": 0
  }
]
```

POST /api/Users

The screenshot displays the Postman interface for a POST request to `http://localhost:3000/api/Users/`. The request body is a JSON object with the following fields:

```
1 {
2   "Id": 1,
3   "UserName": "tieu",
4   "Password": "qwerty",
5   "Email": "tieu@aol.com"
6 }
```

The response is a 201 Created status, with a response time of 102 ms and a body size of 383 B. The response body is shown in the "Body" tab, formatted as JSON:

```
1 {
2   "Id": 1,
3   "UserName": "tieu",
4   "Password": "qwerty",
5   "Email": "tieu@aol.com",
6   "_id": "674d661d9af6fa1fbb90a2ae",
7   "__v": 0
8 }
```

The interface includes a sidebar with "Collections", "Environments", "History", and "API Network". The top bar shows the current workspace and request details. The bottom status bar includes options like "Online", "Find and replace", "Console", "Postbot", "Runner", "Start Proxy", "Cookies", "Vault", "Trash", and a help icon.

GET /api/Users/{id}

The screenshot shows the Postman application interface. The top bar includes navigation icons, a search bar, and an 'Upgrade' button. The left sidebar contains 'Collections', 'Environments', 'History', and a workspace icon. The main area displays a request to `http://localhost:3000/api/Users/3` using the `GET` method. The 'Body' tab is selected, showing a message: 'This request does not have a body'. The response is displayed below, showing a status of `200 OK`, a time of `91 ms`, and a size of `409 B`. The response body is formatted as JSON and shows the following data:

```
1 {
2   "_id": "674d66739af6fa1fbb80a2b2",
3   "Id": 3,
4   "UserName": "nguyen",
5   "Password": "IKnowThisIsNotAStrongPassword",
6   "Email": "nguyen@outlook.com",
7   "__v": 0
8 }
```

The bottom status bar includes icons for 'Console', 'Postbot', 'Runner', 'Vault', and other utility tools.

PUT /api/Users/{id}

The screenshot shows the Postman application interface. At the top, there's a header with 'Workspaces' and 'More' dropdowns, a search bar, and a status bar. The main area is divided into several sections:

- Request Section:** The URL is `http://localhost:3000/api/Users/2`. The method is `PUT`. There are buttons for 'Save' and 'Share'.
- Body Section:** The 'Body' tab is selected. The request body is a JSON object:

```
{  "Email": "changedEmail@mail.ru"}
```

. The format is set to 'JSON'.
- Response Section:** The response status is `200 OK`. The response body is a JSON object:

```
{  "_id": "674d664a9af6fa1fbb80a2b0",  "Id": 2,  "UserName": "phuong",  "Password": "iloveyou123",  "Email": "changedEmail@mail.ru",  "__v": 0}
```

. The format is set to 'Pretty'.

The bottom of the application shows a 'Console' tab and a 'Postbot' button.

DELETE /api/Users/{id}

The screenshot shows a REST client interface with a dark theme. The top bar includes 'Workspaces' and 'More' dropdowns, a search icon, a user icon, a settings icon, a bell icon, a Postman logo, and an 'Upgrade' button. Below this is a breadcrumb trail: '< Overview GET Get ai GET https GET https:'. The main area shows a request to 'http://localhost:3000/api/Users/2' with a 'DELETE' method. The 'Send' button is highlighted in yellow. Below the request bar are tabs for 'Params', 'Auth', 'Headers (8)', 'Body', 'Scripts', and 'Settings'. The 'Body' tab is selected, showing a '200 OK' status with a response time of 97 ms and a body size of 293 B. The response is displayed in 'Pretty' format as a JSON object:

```
{  "message": "User deleted"}
```

. The bottom status bar includes icons for 'Console', 'Postbot', 'Runner', 'Vault', and other utility icons.

Workspaces ▾ More ▾

< Overview GET Get ai GET https GET https:

http://localhost:3000/api/Users/2 Save ▾ Share </>

DELETE ▾ http://localhost:3000/api/Users/2 Send ▾

Params Auth Headers (8) Body • Scripts Settings Cookies

Query Params

	Key	Value	Description	...	Bulk Edit
	Key	Value	Description		

Body ▾ ↻

200 OK • 97 ms • 293 B • 🌐 | 📄 ⋮

Pretty Raw Preview Visualize JSON ▾ ⌵

```
1 {
2   "message": "User deleted"
3 }
```

📄 🔍 Console Postbot Runner ⌵ ⌚ Vault 🗑️ ⌵ ?

THE END
